Controlador de semáforo

Herrera Mauro Leandro

Resumen

Este informe describe el diseño, la descripción en hardware (VHDL) y la simulación de un controlador lógico para un semáforo de intersección. El controlador está pensado para funcionar en una placa EDU-CIAA-FPGA. El sistema maneja la secuencia estándar de luces. El rojo dura 60 segundos, el verde 50 segundos y el amarillo 10 segundos. También incluye funciones extra de interrupción con sensores para vehículos de emergencia y botones para el cruce peatonal. El sistema cumple reglas estrictas de prioridad, extensión de tiempos e interbloqueo. Para resolver este problema, se usan y comparan dos métodos de diseño de circuitos secuenciales. Uno es una Máquina de Estado Finito (FSM) monolítica. El otro es una arquitectura que separa la lógica de control de la ruta de datos, como temporizadores y registros. Al final, se muestran los diagramas de estado, la síntesis de los circuitos y la validación con bancos de pruebas para situaciones normales, peatonales y de emergencia.

1. Introducción

El diseño de sistemas de control de tráfico es un ejemplo clásico para la aplicación de lógica secuencial sincrónica y máquinas de estado finito (FSM). El presente proyecto tiene como objetivo diseñar, simular e implementar en hardware (mediante lenguaje VHDL y una placa EDU-CIAA-FPGA) un controlador lógico para un semáforo ubicado en la intersección de dos calles.

Para garantizar un flujo de tráfico seguro y ordenado, el sistema debe cumplir con un ciclo de funcionamiento normal y responder a eventos externos (como peatones o emergencias) bajo las siguientes especificaciones técnicas:

1. Secuencia y Temporización Normal:

- El ciclo estándar de luces para cualquier calle es: Verde (50 segundos), Amarillo (10 segundos) y Rojo (60 segundos).
- Por seguridad, el sistema debe garantizar que, mientras una calle se encuentre en estado Verde o Amarillo, la calle perpendicular debe permanecer obligatoriamente en estado Rojo.

2. Modo Peatonal (Interrupción Temporal):

- Cada calle cuenta con un pulsador para el cruce de peatones.
- Al activarse, el sistema extiende al doble el tiempo de la luz verde correspondiente (es decir, el verde pasa de 50 s a 100 s).

- Este modo goza de máxima prioridad sobre la temporización: si la calle contraria entra en modo de emergencia mientras el cruce peatonal está activo, el sistema no abortará el cruce. Esperará a que finalice el tiempo peatonal extendido antes de realizar la transición a amarillo.

3. Modo de Emergencia (Interrupción de Alta Prioridad):

- Cada dirección posee un sensor que detecta la aproximación de vehículos de servicio de emergencia (ambulancias, bomberos, etc.).
- Si una calle está en **rojo** y su sensor de emergencia se activa, el sistema fuerza a la calle contraria (que está en verde) a pasar inmediatamente a amarillo, cortando su tiempo normal, para luego ceder el paso (verde) a la calle en emergencia.
- Si una calle ya está en **verde** y su sensor se activa, el sistema "congela" el estado verde indefinidamente hasta que el vehículo haya pasado y el sensor se desactive.
- Existe un mecanismo de interbloqueo (*lockout*): si una calle ya se encuentra en modo de emergencia, la calle contraria no puede entrar en dicho modo hasta que la primera emergencia haya sido despejada.

4. Interfaz de Hardware:

- El diseño incluye una entrada de reinicio asincrónico activo en bajo (nreset) para inicializar el sistema.
- La base de tiempos se generará a partir del reloj de 12 MHz integrado en la FPGA.
- Las salidas visuales se mapearán a los segmentos de un display, y las entradas (sensores y pulsadores) se emularán utilizando los interruptores físicos de la placa según el mapa de bits establecido.

2. Materiales y Métodos

El diseño de un controlador de semáforo con múltiples temporizaciones y prioridades cruzadas requiere un enfoque sistemático basado en lógica secuencial sincrónica. Para su resolución, se estudiaron y aplicaron dos metodologías de diseño distintas: la Máquina de Estado Finito única y la Máquina de Estado Finito factorizada.

2.1 Diseño mediante Máquina de Estado Finito (FSM) Única Una Máquina de Estado Finito (FSM, por sus siglas en inglés) es un modelo matemático y de hardware que consta de un registro que almacena el estado actual del sistema, un bloque de lógica combinacional que calcula el estado siguiente y otro bloque combinacional que determina las salidas. Según cómo se generen estas salidas, la FSM puede clasificarse en una máquina de *Moore* (las salidas dependen únicamente del estado actual) o de *Mealy* (las salidas dependen del estado actual y de las entradas concurrentes).

El diseño estructurado de una FSM única sigue un procedimiento riguroso:

1. **Diagrama de Transición de Estados:** Se identifican las entradas y salidas, y se dibuja un grafo indicando todos los estados posibles del sistema y las condiciones lógicas de transición entre ellos. En este enfoque, el tiempo y los eventos externos se resuelven dentro del mismo diagrama.
2. **Codificación de Estados:** Se asignan valores binarios a cada estado. La elección de la codificación (binaria pura o *one-hot*) impacta directamente en el tamaño del hardware resultante.
3. **Tablas de Verdad:** Se elaboran las tablas de transición de estado y de salidas.

4. **Ecuaciones y Síntesis:** Se obtienen las ecuaciones booleanas minimizadas y se modela la arquitectura en lenguaje de descripción de hardware (VHDL).

La limitación de este enfoque monolítico es que, ante secuencias complejas o tiempos de espera prolongados, el diagrama de estados puede volverse denso, engorroso y propenso a errores humanos de diseño.

2.2 Diseño mediante Máquinas de Estado Factorizadas (Ruta de Datos y Control)

El diseño de FSMs complejas a menudo se simplifica drásticamente si el sistema se divide en múltiples máquinas de estado más simples que interactúan entre sí, de modo que la salida de una máquina funciona como entrada de otra. A esta aplicación de jerarquía y modularidad se la denomina **factorización** de máquinas de estado.

En la práctica, la factorización divide el problema en dos grandes dominios:

- **Ruta de Datos (Datapath) o Subsistemas Periféricos:** Se encarga de manejar variables repetitivas o tareas específicas. En el caso del semáforo, estos módulos periféricos son los *temporizadores* (que cuentan los pulsos de reloj hasta alcanzar los 10, 50 o 60 segundos) y las *memorias de cruce peatonal* (registros que guardan temporalmente si un peatón presionó el botón).
- **Lógica de Control Principal:** Es una FSM mucho más reducida cuya única tarea es tomar decisiones. En lugar de contar segundos, el controlador simplemente espera una señal de "tiempo cumplido" proveniente del temporizador, o una bandera de "peatón esperando" desde el registro.

Este enfoque modular resulta sumamente eficiente. Al extraer los contadores de tiempo y las memorias de eventos del diagrama principal, la FSM de control se vuelve abstracta, limpia y muy fácil de codificar en VHDL.

3. Resultados

3.1 Diseño mediante Máquina de Estado Única

En el primer enfoque, se diseñó una FSM monolítica que gestiona tanto los cambios de luces como las extensiones peatonales y las interrupciones de emergencia en un único bloque de control.

A partir de los diagramas de estado desarrollados, el sistema cuenta con 6 estados principales:

- **RV (Rojo-Verde):** Calle 1 en Rojo, Calle 2 en Verde.
- **RVEXT:** Extensión del tiempo verde por cruce peatonal.
- **RA (Rojo-Amarillo):** Transición de precaución.
- **VR (Verde-Rojo):** Calle 1 en Verde, Calle 2 en Rojo.
- **VREXT:** Extensión del tiempo verde por cruce peatonal.
- **AR (Amarillo-Rojo):** Transición de precaución.

El diseño estructural en VHDL de esta arquitectura emplea señales de bandera (Z) provenientes de un contador genérico para gestionar los tiempos, y prioriza la interrupción de emergencia evaluando las entradas E_EO y E_NS dentro de la lógica del estado siguiente. Para mantener la concisión de este informe, el código fuente completo del

módulo ha sido versionado y se encuentra alojado para su revisión en el repositorio de GitHub correspondiente a esta actividad.

3.2 Simulación y Validación (Máquina Única)

Para verificar la correctitud del diseño lógico, se desarrolló un banco de pruebas (*Testbench*) exhaustivo que estimula las entradas de la FSM. Dado que la temporización real en segundos requeriría una simulación de millones de ciclos de reloj, el entorno de prueba manipula directamente la bandera de fin de conteo (Z) para evaluar las transiciones lógicas puras.

El banco de pruebas fue diseñado para cumplir estrictamente con los tres escenarios exigidos en las especificaciones:

1. **Funcionamiento básico:** Se comprobó la secuencia ininterrumpida RV→RA→VR→AR→RV, alternando correctamente las señales de luces (L_NS y L_EO) y cargando los valores de temporización correspondientes en Q_max (50 s o 10 s).
2. **Caso de cruce peatonal:** Estando en estado verde (RV), se activó el pulsador S_EO. Al finalizar el tiempo normal, se verificó que el sistema transita al estado de extensión (RVEXT), emite la señal de limpieza de memoria (RES_EO = '1') y duplica el tiempo antes de pasar a amarillo.
3. **Caso de emergencia (Interrupción asincrónica):** Estando la calle Este-Oeste en rojo, se activó el sensor de emergencia E_EO. Se verificó que el sistema aborta la espera de la bandera Z y fuerza una transición inmediata al estado de precaución

(AR), para luego otorgar luz verde a la calle en emergencia, respetando las prioridades de interbloqueo.

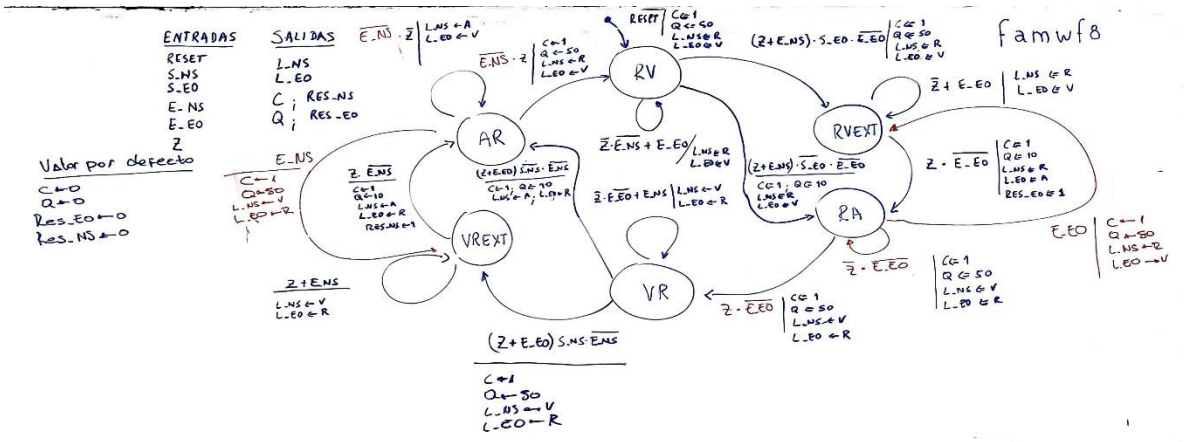


Figura 1. *Diagrama de transición de estados para el controlador del semáforo. Se representan los estados de la secuencia normal y las condiciones lógicas de transición basadas en las señales de los temporizadores y los sensores externos.*

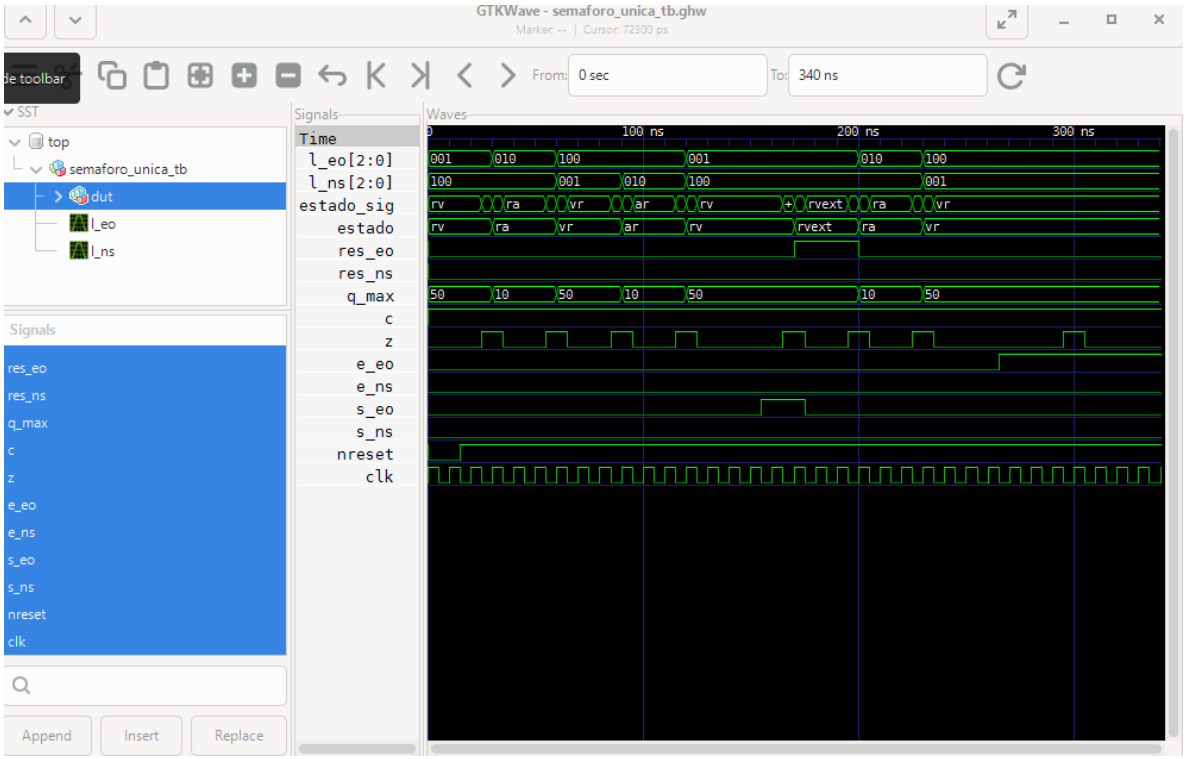


Figura 2. *Resultados de la simulación temporal de la FSM única en GTKWave. Se muestran las formas de onda correspondientes al ciclo normal, la extensión del cruce peatonal y la interrupción asincrónica por emergencia.*

3.3 Diseño mediante Máquina Factorizada

En el segundo enfoque, se implementó la arquitectura modular detallada en los diagramas de bloques. En lugar de procesar todas las señales en un solo bloque, el sistema se dividió en componentes de hardware dedicados (Ruta de Datos) y un controlador principal (Lógica de Control):

- **Memoria de Cruce (memoria_peaton):** Dos biestables (Norte-Sur y Este-Oeste) que registran asincrónicamente si un peatón presionó el botón, manteniendo la señal activa (S_NS o S_EO) hasta que el controlador principal acusa recibo y las reinicia (RES_NS, RES_EO).
- **Temporizador Genérico:** Un contador sincrónico que recibe un valor de carga (Q_max) y una señal de habilitación (C). Al alcanzar el tiempo objetivo, emite la bandera Z.
- **Controlador (FSM Principal):** Una máquina de estados abstracta. Al delegar el conteo de tiempo y la retención de los botones a los módulos periféricos, el diagrama de estados del controlador se simplificó enormemente. Sus transiciones ahora dependen únicamente de banderas booleanas (Z, S, E), haciendo que el código sea más legible y menos propenso a errores de temporización.

Al igual que con la máquina única, el código fuente VHDL de esta arquitectura estructurada (Top-Level y submódulos) se encuentra alojado en el repositorio de GitHub adjunto a esta entrega. La simulación de este módulo arroja exactamente el mismo diagrama de tiempos que la Figura 1, demostrando que ambos enfoques lógicos son funcionalmente equivalentes desde la perspectiva del usuario.

4. CONCLUSIONES

El desarrollo de este controlador de semáforo permitió contrastar de forma práctica dos metodologías fundamentales en el diseño de sistemas digitales.

Por un lado, el diseño mediante una **FSM Única** resulta ser un enfoque directo y fácil de implementar cuando el número de variables es bajo. Sin embargo, su principal desventaja quedó en evidencia al incorporar los retardos largos y las interrupciones peatonales: la FSM debe gestionar simultáneamente el estado de las luces, las transiciones y los tiempos, lo que genera ecuaciones combinacionales densas y diagramas difíciles de escalar.

Por otro lado, la **FSM Factorizada (Datapath + Control)** demostró ser una solución técnica superior para este tipo de problemas. Al separar las tareas repetitivas (como el conteo del temporizador y la retención de los pulsadores) de la toma de decisiones, la lógica de control se vuelve abstracta y modular. Las ventajas de este enfoque son claras: facilita la depuración, permite reutilizar módulos estándar (temporizadores) en futuros proyectos, y hace que el sistema sea fácilmente escalable (por ejemplo, si en el futuro se deseara agregar un tercer semáforo o un sensor de velocidad). En conclusión, aunque la factorización requiere planificar la interconexión de módulos (Top-Level) al inicio del proyecto,

5. REFERENCIAS

- Floyd, T. L. (2000). *Fundamentos de Sistemas Digitales* (7.ª ed.). Prentice Hall.

  Fundamentos de Sistemas Digitales - Con CD 7 Ed (Spanish -- Thomas L. Floyd -- (WeLib.org).pdf

- Harris, D., & Harris, S. L. (2007). *Digital Design and Computer Architecture*. Morgan Kaufmann.

  Digital_Design_and_Computer_Architecture.pdf

- Mano, M. M. (2003). *Diseño digital* (3.ª ed.). Pearson Educación.

  Diseño digital [recurso electrónico -- M. Morris Mano; Michael D. Ciletti -- (WeLib.org).pdf

- Wakerly, J. F. (2001). *Diseño digital: principios y prácticas* (3.ª ed.). Pearson Educación.